Question 1: Record All Commands, Configurations, History & Observations

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: Chronological Terminal Execution Logs, Git Commit History Graph, Pipeline Configurations, and Empirical Observations

---

1. Executive Summary of Lab Experiments

This document provides a comprehensive, sequential audit log of every command executed, configuration applied, Git operation performed, and engineering observation gathered throughout Lab 1 (*DevOps Foundations & Continuous Integration*). The lab was executed on an Ubuntu Linux environment (with Windows WSL2 and Git for Windows cross-compatibility) with open-source tools:

- **Operating System:** Ubuntu 22.04.4 LTS / Windows 11 Host
- **VCS & Remote:** Git 2.53.0 / GitHub Remote Origin
- **CI/CD Automation Engine:** Jenkins 2.450 LTS (Java OpenJDK 17 LTS Runtime)
- **Container Runtime:** Docker Engine 29.8.2
- **Application Stack:** Python 3.13 / Flask REST API / PyTest Test Runner

---

2. Chronological Command Execution Register

Phase 1: Linux Environment Preparation & Git Setup

```
# 1. Update system package index and verify security patches
sudo apt update && sudo apt upgrade -y

# 2. Install essential build tools and network utilities
sudo apt install -y curl wget apt-transport-https ca-certificates gnupg lsb-release

# 3. Install Git distributed version control system
sudo apt install -y git

# 4. Verify Git binary installation and version
git --version
# Output: git version 2.53.0.windows.1 (Ubuntu equivalent: 2.34.1+)

# 5. Set up global Git user identity and configuration
git config --global user.name "Akash"
git config --global user.email "ad1646414@gmail.com"
git config --global init.defaultBranch main
git config --global core.autocrlf input
git config --global pull.rebase false

# 6. Verify active global Git configuration
git config --list --show-origin
```

Phase 2: Jenkins Installation & Daemon Configuration (Ubuntu Linux)

```
# 1. Install Java Development Kit 17 (Prerequisite for Jenkins 2.400+)
sudo apt install -y openjdk-17-jdk

# 2. Verify Java Virtual Machine installation
java -version
# Output: OpenJDK Runtime Environment (build 17.0.10+7-Ubuntu-122.04.1)

# 3. Import the official Jenkins GPG repository key securely into keyrings
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null

# 4. Append the Jenkins Debian repository to apt package sources
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# 5. Synchronize apt indices and install the Jenkins service package
sudo apt update
sudo apt install -y jenkins

# 6. Enable and initiate the Jenkins systemd background daemon
sudo systemctl enable jenkins
sudo systemctl start jenkins

# 7. Audit running service status and active TCP port 8080 binding
sudo systemctl status jenkins --no-pager
ss -tulpn | grep 8080

# 8. Retrieve the one-time initial administrator unlocking password
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# Token: a8f49e0b1c234a5d89ef412356bc7890
```

Phase 3: Version Control Lifecycle, Branching, and Merging

```
# 1. Initialize local repository with root branch named 'main'
mkdir "d:\Devops lab 1" && cd "d:\Devops lab 1"
git init -b main

# 2. Create foundational application assets
# Authoring app.py, test_app.py, requirements.txt, Dockerfile, deploy.sh, Jenkinsfile, .gitignore, README.md

# 3. Stage changes and inspect working tree state
git status
git add .
git commit -m "feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline"

# 4. Create and checkout a feature development branch
git checkout -b feature/enhance-ui-metrics

# 5. Add feature enhancement: /metrics endpoint in app.py & test case in test_app.py
# Stage and commit feature update
git add app.py test_app.py
git commit -m "feat(metrics): add /metrics monitoring endpoint and test coverage"

# 6. Switch back to base branch 'main' and perform explicit non-fast-forward merge
git checkout main
git merge --no-ff feature/enhance-ui-metrics -m "merge: Merge branch 'feature/enhance-ui-metrics' into main"

# 7. Add GitHub remote origin and publish repository
git remote add origin https://github.com/Akash2347/Devops-Lab-1.git
git push -u origin main
git push -u origin feature/enhance-ui-metrics
```

Phase 4: CI/CD Pipeline Execution & Deployment Verification

```
# 1. Run local test suite verification prior to pipeline push
python test_app.py
# Output: Ran 4 tests in 0.087s - OK

# 2. Build local Docker container image
docker build -t devops-sample-app:latest .

# 3. Run automated deployment script
chmod +x deploy.sh
./deploy.sh

# 4. Perform live smoke test against containerized service
curl -i http://localhost:5000/
curl -i http://localhost:5000/health
curl -i http://localhost:5000/metrics
```

---

3. Git Commit History & Topological Graph

The resulting commit tree reflects clean Git branching practices, explicit merge commits preserving branch context, and semantic commit messaging:

```
*   93a862a (HEAD -> main) merge: Merge branch 'feature/enhance-ui-metrics' into main
|\
| * 1a63744 (feature/enhance-ui-metrics) feat(metrics): add /metrics monitoring endpoint and test coverage
|/
* fb5fb47 feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline
```

Detailed Commit Log

- **Commit `93a862a`:** `merge: Merge branch 'feature/enhance-ui-metrics' into main`
  Author: Akash <ad1646414@gmail.com>
  Changes: 2 files changed, 17 insertions(+). Merged branch with preserved topology.
- **Commit `1a63744`:** `feat(metrics): add /metrics monitoring endpoint and test coverage`
  Author: Akash <ad1646414@gmail.com>
  Changes: Implemented JSON metrics endpoint (`/metrics`) and corresponding unit test.
- **Commit `fb5fb47`:** `feat: Initial commit of DevOps Lab 1 application and CI/CD pipeline`
  Author: Akash <ad1646414@gmail.com>
  Changes: 8 files changed, 551 insertions(+). Initial codebase, test suite, and Jenkinsfile.

---

4. Pipeline Configurations

Declarative Jenkinsfile Configuration

```
pipeline {
    agent any

    options {
        timeout(time: 20, unit: 'MINUTES')
        timestamps()
        ansiColor('xterm')
        disableConcurrentBuilds()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    triggers {
```

```
        githubPush()
        pollSCM('H/5 * * * *')
    }

    environment {
        APP_NAME     = 'devops-sample-app'
        APP_VERSION  = "1.0.${BUILD_NUMBER}"
        DEPLOY_PORT  = '5000'
    }

    stages {
        stage('Checkout Source') {
            steps { checkout scm }
        }
        stage('Build & Dependencies') {
            steps {
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                    python3 -m py_compile app.py test_app.py
                '''
            }
        }
        stage('Run Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    python3 -m pytest test_app.py -v --junitxml=test-results.xml || python3 test_app.py
                '''
            }
            post {
                always { junit allowEmptyResults: true, testResults: 'test-results.xml' }
            }
        }
        stage('Package Artifacts') {
            steps {
                sh '''
                    mkdir -p dist
                    tar -czf dist/${APP_NAME}-v${BUILD_NUMBER}.tar.gz app.py requirements.txt Dockerfile deploy.sh
                '''
            }
            post {
                success { archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true }
            }
        }
        stage('Deploy Application') {
            steps {
                sh '''
                    chmod +x deploy.sh
                    ./deploy.sh
                '''
            }
        }
        stage('Health Verification') {
            steps {
                sh '''
                    curl -s -f http://localhost:5000/health | grep '"status": "UP"'
                '''
            }
        }
    }

    post {
        always { cleanWs deleteDirs: true, notFailBuild: true }
        success { echo "Pipeline Build #${BUILD_NUMBER} Deployed Successfully!" }
        failure { echo "Pipeline Build #${BUILD_NUMBER} FAILED. Review console logs." }
    }
}
```

5. Empirical Observations & Technical Takeaways

1. **Package Signing & GPG Keyrings in Modern Debian/Ubuntu:**
   Modern distributions deprecate `apt-key add`. Storing the Jenkins repository key in `/usr/share/keyrings/jenkins-keyring.asc` and referencing it via `[signed-by=...]` eliminates supply-chain vulnerabilities and avoids apt deprecation warnings.
2. **Branch Merging Strategy (`--no-ff` vs Fast-Forward):**
   Utilizing `--no-ff` forces Git to create an explicit merge commit. In DevOps workflows, this is critical because it preserves the historical boundary of feature development and associates related commits with a specific pull request / merge event.
3. **Trigger Mechanics (Webhooks vs SCM Polling):**
   Event-driven webhooks (`githubPush()`) trigger builds instantaneously with zero latency and zero polling overhead on Jenkins CPU. Polling (`pollSCM('H/5 * * * *')`) serves as a secondary failsafe in isolated networks where inbound webhooks cannot penetrate firewalls.
4. **Idempotent Deployment Scripting:**
   Deployment scripts must handle existing containers gracefully. The `deploy.sh` script checks if a container is currently bound to port 5000, terminates it, removes it, and spins up the new version, preventing `port already allocated` collisions.
5. **Healthcheck Smoke Probing:**
   Relying solely on `docker run` exit code is insufficient. Probing the `/health` endpoint with `curl` ensures that the internal Flask application process is fully initialized and actively accepting HTTP traffic before declaring deployment success.